

Lab 6 — Analog-ish Output: PWM with the LEDC Peripheral

ESP32-S3 FIRMWARE LAB SERIES · PHASE 1 · FOUNDATIONS

Estimated time	2 hrs
Introduces	duty cycle, resolution/frequency trade-off, double buffering
Prerequisites	Labs 1 and 4.
Beyond the board	Optional: an LED and a 330 Ω resistor. A logic analyzer to verify duty.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§6.1 objective**, **§6.2 parts and wiring**, **§6.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

6.1 Objective & TM4C bridge

Produce a smoothly varying output from a chip that can only drive a pin fully on or fully off, and understand the two-part timer-and-channel architecture that every PWM peripheral shares.

TM4C BRIDGE

The bare-metal PWM lab has you configure a PWM generator block, set the period in the `LOAD` register and the duty in `CMPA / CMPB`, and use the update registers so a change takes effect cleanly at a period boundary. Every one of those concepts reappears here: `LOAD` becomes frequency-plus-resolution, `CMPA` becomes duty, and the update register becomes `ledc_update_duty`. Same silicon idea, different names.

6.2 Parts & wiring

Item	Status	Notes
ESP32-S3 board	Required	—
LED + 330 Ω resistor	Optional	Makes the result visible. The lab works measured on the analyzer alone.

Item	Status	Notes
Logic analyzer	recommended	CH0 to GPIO4, 4 MHz, 1 M samples.

From	To
GPIO4	330 Ω resistor, then LED anode (long leg)
LED cathode (short leg, flat side of the rim)	GND

An LED needs a **current-limiting resistor** in series. Without one it draws more current than the pin can safely supply and you risk damaging the LED, the pin, or both. LEDs are also polarised: fitted backwards they simply do not light, so if nothing happens, flip it before you start debugging code.

6.3 How it works

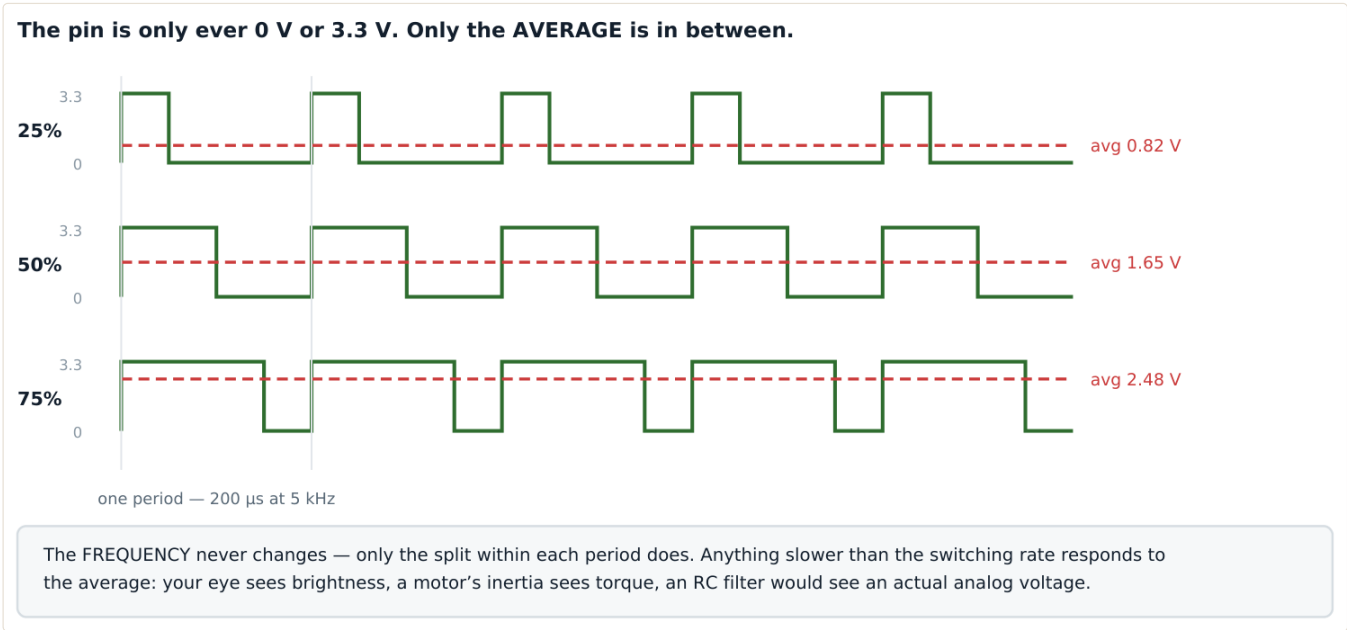


Figure 6.1. The same 5 kHz frequency at three duty cycles. The pin is only ever 0 or 3.3 V; the dashed line is the average, which is what anything slower than the switching rate actually responds to.

PWM duty cycle at 25, 50 and 75 percent, and the resulting average voltage.

The pin is only ever 0 V or 3.3 V. Only the AVERAGE is in between.

Duty	Average voltage
25%	0.83 V
50%	1.65 V
75%	2.48 V

All three waveforms have the same period -- 200 microseconds at 5 kHz. The **FREQUENCY** never changes; only the split within each period does.

Anything slower than the switching rate responds to the average: your eye sees brightness, a motor's inertia sees torque, and an RC filter would see an actual analog voltage.

You cannot output 1.6 volts, so you cheat with time

A digital output pin has two states: 0 V and 3.3 V. There is no built-in way to output half a volt. But if you switch the pin on and off *fast enough*, the average voltage over time is anything you want, and any system slower than the switching rate responds to that average rather than the individual pulses. Your eye cannot follow a 5 kHz flicker, so it perceives brightness. A motor's inertia cannot follow it either, so it perceives torque.

The fraction of each period spent HIGH is the **duty cycle**. Twenty-five percent duty at 3.3 V averages 0.83 V; seventy-five percent averages 2.5 V. The **frequency** stays fixed; only the split within each period moves. That is the entire idea, and it is why PWM is described as "analog-ish" rather than analog — the instantaneous voltage is always 0 or 3.3, and only the average is intermediate.

Choosing the frequency is a real decision. Too low and you see flicker or hear audible whine from a motor. Too high and switching losses rise and the resolution ceiling drops (below). 5 kHz is a comfortable default for an LED.

Why LEDC is a peripheral, not a library

The name is unfortunate: **LEDC** stands for LED Control, but it is a general-purpose hardware PWM generator that happens to have been named after its original use case. It is real silicon, not a software loop. You could in principle toggle a pin from a timer callback and call it PWM, and conceptually that works — but under an RTOS it is strictly worse: it costs CPU on every edge, and any scheduling delay jitters your duty cycle. Hardware PWM runs with zero CPU involvement and no jitter. That comparison is a good, concrete answer to "why use a peripheral instead of software."

The chip also has **MCPWM**, a more elaborate motor-control PWM with dead-time insertion and fault inputs, and **RMT**, which you met in Lab 1. Knowing there are three PWM-capable peripherals and roughly what each is for is worth a sentence in an interview.

The timer/channel split, and why both exist

Configuration comes in two structs and people reasonably ask why. The **timer** defines the shared time base: frequency and resolution. The **channel** binds one output pin to one timer and holds that pin's duty. The relationship is many-to-one: several channels can share a single timer, so four LEDs can run at the same 5 kHz with four completely independent brightnesses. Separating them is what makes that possible with one timer's worth of hardware.

Resolution and frequency are in tension

`LEDC_TIMER_13_BIT` means the duty is divided into $2^{13} = 8192$ steps, so the value runs 0–8191. More bits gives smoother fades; fewer gives coarser ones. The catch is that the peripheral counts a source clock, and to get N steps at frequency f it needs a clock of at least $N \times f$. At 13-bit and 5 kHz that is about 41 MHz, comfortably available. Ask for 13-bit at 50 kHz and you need 410 MHz, which the source clock cannot supply — so the configuration call **fails**. If you find your LED not fading, check that you typed 5000 and not 50000, and check the return code.

WARNING**Check your return codes — this lab is the object lesson**

`ledc_timer_config` and `ledc_channel_config` both return `esp_err_t`. The listing below does not check them, which is a deliberate weakness you should fix in your own copy by wrapping both in `ESP_ERROR_CHECK()`. An invalid frequency/resolution pair fails *silently* otherwise, and you will spend an hour on your wiring before suspecting the config. Checking return codes is the single cheapest professional habit in firmware.

Why setting the duty takes two calls

`ledc_set_duty` stages a new value; `ledc_update_duty` makes it take effect. This is not clumsy API design — it is **double buffering in silicon**. The new duty is latched into the active register at a period boundary, so the waveform never changes mid-pulse. Without that, a duty change landing halfway through a period would emit one malformed pulse: a visible glitch on an LED, and a genuine current spike on a motor. The split also lets you stage several channels and apply them together atomically. **TM4C bridge:** this is precisely the TM4C's `CMPAUPD` update register, and the same stage-then-apply pattern as Lab 1's LED refresh and Lab 10's NVS commit.

6.4 Procedure & code

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/ledc.h"

#define LED_PIN 4

void app_main(void) {
    ledc_timer_config_t t = {
        .speed_mode    = LEDC_LOW_SPEED_MODE,
        .timer_num     = LEDC_TIMER_0,
        .duty_resolution = LEDC_TIMER_13_BIT,    // duty range 0..8191
        .freq_hz       = 5000,
        .clk_cfg       = LEDC_AUTO_CLK,
    };
    ledc_timer_config(&t);                      // TODO: wrap in ESP_ERROR_CHECK -- see 6.3

    ledc_channel_config_t c = {
        .gpio_num      = LED_PIN,
        .speed_mode    = LEDC_LOW_SPEED_MODE,
        .channel       = LEDC_CHANNEL_0,
        .timer_sel     = LEDC_TIMER_0,
        .duty          = 0,
        .hpoint        = 0,
    };
    ledc_channel_config(&c);

    while (1) {
        for (int d = 0; d <= 8191; d += 128) {
            ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0, d);
            ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0); // takes effect on update
            vTaskDelay(pdMS_TO_TICKS(15));
        }
    }
}
```

```
}
}
```

6.5 Code walkthrough, line by line

Thirty-four lines: two config structs and a loop. Every field in those structs corresponds to something you set by hand on the TM4C.

Line 9 · `.speed_mode = LEDC_LOW_SPEED_MODE`

The LEDC peripheral has two independent speed groups. Low-speed is correct here: it supports more clock sources and, on some chips, hardware fading. High-speed mode exists on the original ESP32 but not on the S3, so this is effectively the only choice — worth knowing when you read code written for a different chip.

Line 10 · `.timer_num = LEDC_TIMER_0`

Picks which of the peripheral's four timers to configure. Four timers, eight channels — that ratio is the whole reason the configuration is split in two, as §6.3 explained.

Line 11 · `.duty_resolution = LEDC_TIMER_13_BIT`

How finely the duty cycle can be divided: $2^{13} = 8192$ steps, so the duty value runs **0 to 8191**. More bits gives smoother fades and fewer gives coarser ones, but the number is not free — see the next line.

Line 12 · `.freq_hz = 5000`

5 kHz, far above what the eye can follow, so the LED shows a steady average rather than flicker.

WARNING

This line and the previous one constrain each other

The peripheral produces N steps at frequency f by counting a source clock, so it needs a clock of at least $N \times f$. At 13-bit and 5 kHz that is $8192 \times 5000 \approx 41$ MHz, comfortably available. Ask for 13-bit at **50** kHz and you need 410 MHz, which no source can supply — so `ledc_timer_config` **fails**. If your LED does not fade, check you typed `5000` and not `50000`.

Line 13 · `.clk_cfg = LEDC_AUTO_CLK`

Lets the driver choose a source clock that can actually deliver the requested frequency and resolution — which is precisely the calculation that fails when you ask for too much. You can pin a specific source instead when you need the timing to survive a clock-frequency change or light sleep.

Line 15 · `ledc_timer_config(&t);`

Applies the timer configuration — and note the `TODO`. This call returns `esp_err_t` and the listing ignores it, which is a deliberate weakness for you to fix. Wrap it in `ESP_ERROR_CHECK()` in your own copy. An invalid frequency/resolution pair fails **silently** otherwise, and you will spend an hour suspecting your wiring. **TM4C bridge:** this struct is your PWM `LOAD` register value, expressed as "how fast" plus "how many steps" instead of a raw count.

Line 18 · `.gpio_num = LED_PIN`

The output pin, routed through the GPIO matrix like everything else on this chip.

Line 20 · `.channel = LEDC_CHANNEL_0`

One of eight independent output channels. Each channel has its own pin and its own duty, which is how you drive several LEDs at different brightnesses.

Line 21 · `.timer_sel = LEDC_TIMER_0`

Ties this channel to the timer configured above. **This line is the many-to-one relationship in action:** point four channels at `LEDC_TIMER_0` and all four share its 5 kHz time base while keeping independent duties.

Line 22 · `.duty = 0`

Starts fully off. The valid range is 0 to 8191, set by the 13-bit resolution on line 11 — the two fields are linked, and writing 10000 here would be out of range.

Line 23 · `.hpoint = 0`

Sets *where within each period the pulse begins*. Zero means every channel's pulse starts together, which is the normal default. Staggering hpoints across many channels is a real technique for spreading current draw so all your LEDs do not switch on at the same instant — a nice detail to know exists.

Line 28 · `for (int d = 0; d <= 8191; d += 128)`

Walks the duty from 0 upward in steps of 128, so 64 visible brightness steps. Note it never actually reaches 8191: stepping by 128 from 0 gives 8064 as the last value below the limit, so the LED stops just short of full brightness. A small off-by-one, left in deliberately for you to find and fix in §6.7.

Line 29 · `ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0, d);`

Stages the new duty value. Nothing changes on the pin yet. Note the speed mode and channel are passed on every call rather than being remembered in a handle — a different API style from the handle-based drivers elsewhere in this series, and worth noticing so the inconsistency does not trip you up.

Line 30 · `ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0);`

Applies it. The split is not clumsy API design — it is **double buffering in silicon**. The staged value is latched into the active register at a period boundary, so the waveform never changes mid-pulse. Without that, a duty change landing halfway through a period would emit one malformed pulse: a visible glitch on an LED and a real current spike on a motor.

Forgetting this line is the classic Lab 6 bug. The code runs, no error appears, the LED sits at whatever it was. **TM4C bridge:** this is precisely the TM4C's `CMPAUPD` update register, and the same stage-then-apply pattern as Lab 1's `led_strip_refresh` and Lab 10's `nvs_commit`.

Line 31 · `vTaskDelay(pdMS_TO_TICKS(15));`

Paces the ramp so a full sweep takes about a second (64 steps × 15 ms), and yields the CPU between steps. It also has a consequence for debugging: each step lasts 15 ms, so an analyzer capture window of a few milliseconds sits entirely inside one step and the waveform looks static. Zoom out to a few hundred milliseconds to see the ramp.

Lines 27, 32-33 · the outer while(1)

When `d` passes the limit the inner `for` ends and the outer loop restarts it at 0, which is the sharp snap back to darkness you see. The ramp is a **sawtooth**, not a breath; making it breathe means ramping back down, which is the first exercise in §6.7.

6.6 Docs & references

DOCS

Trace every line to its source

`ledc_timer_config_t`, `ledc_channel_config_t`, `ledc_set_duty`, `ledc_update_duty` — ESP-IDF API Reference → Peripherals → **LED Control (LEDC)**. The "Functional Overview" section explains the timer/channel model and is the direct answer to why there are two configs.

The frequency/resolution constraint — the same page has a section on supported ranges and a formula. Work out for yourself the maximum frequency at 13-bit resolution, then confirm why 50 kHz fails.

`ledc_set_fade_with_time` — also on that page. The peripheral can perform hardware fades with no CPU involvement at all, which makes your ramp loop unnecessary. Worth knowing it exists.

MCPWM — Peripherals → MCPWM, if you want to see what a motor-grade PWM peripheral adds (dead time, fault handling, complementary outputs).

6.7 Verify, troubleshoot, your turn

Verify. The LED ramps from off to bright over about a second, then snaps back. On the analyzer, measure two things: the **period** should be 200 μ s, confirming 5 kHz, and the **high time** at several points in the ramp should match `d/8191` as a percentage, confirming the driver's duty abstraction against the real waveform. Doing that second measurement is what turns "it looks brighter" into engineering.

NOTE

If the trace looks like it is not changing

That is a timescale problem, not a fault. Each duty step lasts 15 ms, and a capture window of a few milliseconds sits entirely inside one step, so the waveform looks static. Zoom out to cover several hundred milliseconds and the ramp becomes obvious. This is a good general lesson about analyzer captures: always ask whether your window spans the phenomenon you are looking for.

Troubleshoot

Symptom	Cause and fix
LED never lights	Fitted backwards, or no resistor path to GND. Check polarity first.
LED is on but does not fade	Missing <code>ledc_update_duty</code> , or the timer config failed silently — check for <code>50000</code> where you meant <code>5000</code> .
Config returns an error	The frequency and resolution pair is unreachable. Lower one of them.

Symptom	Cause and fix
Flat analyzer traces	Missing ground between analyzer and board.

Your turn

1. **Make it breathe.** Ramp up then back down, and fix the off-by-one so it actually reaches 8191.
2. **Add a second channel** on another pin sharing the same timer, running at a different duty. Confirm on two analyzer channels that both have a 200 μ s period but different high times — that is the many-to-one architecture demonstrated on real signals.
3. **Break it deliberately.** Set the frequency to 50 kHz at 13-bit, wrap the config in `ESP_ERROR_CHECK`, and watch it abort with a clear message. Then work out the maximum frequency your resolution allows. Log it — "I hit an invalid PWM config and learned to check return codes" is a real, small, honest war story.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 6.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06 ←	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.